Universidad Nacional de Colombia
Sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Ingeniería de Software 2

Taller 2 – Planeación y estimación, parte 2

Estimación con Planning Poker, capacidad, velocidad y plan de releases

Integrantes:
Nicolás Molano Baron (Product Owner)
Luis Gabriel Peraza Muñoz (Scrum Master)
Rafael Otero Erazo (Developer)
Sebastian Clavijo Tocasuchyl (Developer)
Nicolás Gómez Lozano (Developer)

Docente: Liliana Marcela Olarte Mesa
Fecha: 28 de septiembre de 2026

1. Estimación de las historias de usuario con Planning Poker

La estimación la realizaron los cinco integrantes del equipo sobre las quince historias de usuario que quedaron en la parte 1 del taller, ya corregidas y completadas, usando una baraja de la serie de Fibonacci (1, 2, 3, 5, 8, 13 y 20).

La dinámica fue la misma con cada historia: el Product Owner la leyó y resolvió las dudas; cada integrante escogió su carta en privado y todos las mostraron al mismo tiempo; cuando hubo diferencias, quienes habían votado el valor más alto y el más bajo explicaron por qué, y se volvió a votar hasta llegar al consenso. Ninguna historia necesitó más de dos rondas.

El equipo ancló la escala con una historia de referencia: la HU 1.1 (seleccionar el tamaño de la pizza), que vale 2 puntos y es la más pequeña del backlog. Las demás se estimaron comparándolas con ella y entre sí, no calculando horas. Los puntos miden el esfuerzo total de dejar la historia terminada (análisis, desarrollo, pruebas e integración) e incluyen tanto el volumen de trabajo como su complejidad y su incertidumbre.

Épica 1: Crear un proceso de pedido intuitivo para el cliente

Rondas de Planning Poker (Molano / Peraza / Otero / Clavijo / Gómez):

Historia	Ronda 1	Argumentos de la discusión	Ronda 2
HU 1.1	1 / 2 / 2 / 3 / 2	Quien votó 3 pensaba que había que calcular los precios por tamaño; se aclaró que eso viene del menú (HU 1.5). Es un control de selección y guardar el valor.	2
HU 1.2a	3 / 5 / 5 / 8 / 5	El 8 incluía también editar el carrito, que es la HU 1.2b. El 3 no había contado que el carrito maneja productos de distinto tipo y debe sobrevivir a la navegación.	5
HU 1.2b	3 / 2 / 3 / 3 / 5	Se acordó que, hecha la HU 1.2a, aquí solo se agregan operaciones sobre un estado que ya existe; por eso vale menos que agregar productos.	3
HU 1.3	2 / 3 / 3 / 3 / 5	El 5 venía de suponer descuentos y promociones, que no están en el backlog. Queda en 3 porque hay que contemplar el costo de domicilio de la HU 1.4.	3
HU 1.4	5 / 3 / 5 / 5 / 8	El 8 contaba la validación de la dirección contra un mapa, que pertenece a la HU 3.2. Se deja en 5: formulario, validación propia y una bifurcación que cambia el costo y el flujo del pedido.	5
HU 1.5	3 / 3 / 5 / 2 / 3	Es un catálogo leído de la base de datos, comparable con la HU 1.2b. El trabajo extra está en la presentación, no en la lógica.	3

HU 1.1: Como cliente, quiero seleccionar el tamaño de la pizza (personal, mediana, grande o familiar), para pedir la cantidad justa según cuántas personas vamos a comer.

Puntuación: 2

Justificación: Es una tarea de baja complejidad. Solo requiere implementar controles simples en la interfaz (como botones o listas) y guardar la selección temporalmente.

HU 1.2a: Como cliente, quiero agregar productos (pizzas, bebidas, adicionales) a mi carrito, para armar el pedido completo en una sola compra.

Puntuación: 5

Justificación: Complejidad media. Implica construir la lógica de estado del carrito de compras, crear estructuras de datos para manejar diferentes tipos de productos y retener esa información durante la navegación.

HU 1.2b: Como cliente, quiero quitar productos o cambiar la cantidad de lo que ya tengo en el carrito, para corregir mi pedido si me equivoqué o cambié de opinión.

Puntuación: 3

Justificación: Como la estructura y lógica principal del carrito ya se construye en la HU 1.2a, añadir las operaciones de actualización o eliminación sobre ese estado preexistente representa un esfuerzo menor.

HU 1.3: Como cliente, quiero ver el precio total de mi pedido desglosado (productos, costo de domicilio si aplica y total) antes de pagar, para saber cuánto voy a pagar y no llevarme sorpresas.

Puntuación: 3

Justificación: Solo requiere cálculos matemáticos básicos tomando los valores del carrito, sin mayores desafíos de arquitectura.

HU 1.4: Como cliente, quiero elegir si mi pedido es a domicilio (ingresando mi dirección) o para llevar, para recibirlo de la forma que me quede más cómoda.

Puntuación: 5

Justificación: Implica la creación de formularios de texto, validación de campos para la dirección y el manejo de una lógica condicional que pueda alterar los costos de envío y el flujo del pedido.

HU 1.5: Como cliente, quiero ver el menú con las pizzas disponibles, sus ingredientes y precios, para escoger qué voy a pedir.

Puntuación: 3

Justificación: Se basa en un modelo clásico de consulta a una base de datos para recuperar y mostrar un catálogo en pantalla, siendo una labor rutinaria y directa, la complejidad viene en crear un formato de visualización atractivo y funcional.

Total de la épica: 21 puntos de historia.

Épica 2: Desarrollar un sistema de gestión de pedidos eficiente para el personal

Rondas de Planning Poker (Molano / Peraza / Otero / Clavijo / Gómez):

Historia	Ronda 1	Argumentos de la discusión	Ronda 2
HU 2.1	5 / 8 / 8 / 13 / 8	El 13 se descartó porque el tablero en sí es una lista ordenada; el 5 no contaba el tiempo real. Lo que pesa es la sincronización permanente.	8
HU 2.2	5 / 5 / 3 / 8 / 5	El 8 incluía el envío de la notificación al cliente, que es la HU 3.1. Queda la máquina de estados con sus validaciones y el disparo del evento.	5
HU 2.3	8 / 5 / 5 / 3 / 5	El 8 asumía integrar drivers de impresora térmica. Se acordó generar un PDF imprimible con el formato de la factura, que es lo que baja la estimación.	5

HU 2.1: Como empleado de cocina, quiero ver todos los pedidos pendientes en tiempo real, ordenados por hora de llegada, para prepararlos en orden y que no se nos quede ninguno.

Puntuación: 8

Justificación: Complejidad moderada-alta. Implementar sincronización de datos y colas en "tiempo real" (mediante WebSockets u otras tecnologías) exige un esfuerzo de backend mucho mayor que una página estática.

HU 2.2: Como empleado de la pizzería, quiero actualizar el estado de un pedido (recibido, en preparación, listo para entregar, en camino, entregado), para que el cliente y los domiciliarios sepan en qué va el pedido.

Puntuación: 5

Justificación: Requiere diseñar una máquina de estados, actualizar registros en la base de datos y garantizar que el cambio de estado gatille las notificaciones necesarias al sistema.

HU 2.3: Como cajero, quiero imprimir la factura de un pedido, para entregársela al cliente o pegarla en el empaque del pedido.

Puntuación: 5

Justificación: Conectar la plataforma web con controladores de hardware locales (impresoras térmicas) o generar archivos PDF descargables correctamente formateados introduce un trabajo técnico intermedio.

Total de la épica: 18 puntos de historia.

Épica 3: Implementar un sistema de entrega confiable y rastreable

Rondas de Planning Poker (Molano / Peraza / Otero / Clavijo / Gómez):

Historia	Ronda 1	Argumentos de la discusión	Ronda 2
HU 3.1	3 / 2 / 3 / 5 / 3	El cambio de estado ya lo hace la HU 2.2, así que aquí solo se genera el código y se lanza el aviso. El 5 contaba el envío por SMS, que es la HU 4.3.	3
HU 3.2	13 / 8 / 13 / 20 / 13	Nadie del equipo ha integrado una API de mapas, así que la incertidumbre es alta. Se deja en 13 y se acuerda un spike de investigación antes de planearla; si al empezar el sprint sigue en 13, se divide.	13
HU 3.3	5 / 3 / 5 / 8 / 5	La vista es parecida a la de cocina, pero aceptar un pedido exige control de concurrencia para que dos domiciliarios no tomen el mismo. Por eso no es 3.	5

HU 3.1: Como cliente que eligió la opción de entrega 'Para llevar', quiero recibir una notificación cuando mi pedido esté listo para ser recogido junto con el código de recogida, para acudir al local sin tener que hacer una fila o esperar la preparación del mismo.

Puntuación: 3

Justificación: Generar un código alfanumérico y lanzar una alerta básica en pantalla o por medio del sistema es una funcionalidad acotada y de bajo esfuerzo.

HU 3.2: Como cliente que eligió la opción de entrega 'A domicilio', quiero poder ver la ubicación en tiempo real del domiciliario con mi pedido, en un mapa con la hora de llegada estimada, para saber cuando llegará mi pedido.

Puntuación: 13

Justificación: Esta es una de las tareas más complejas, pues involucra integración con APIs de mapas de terceros (ej. Google Maps), consumo de coordenadas GPS en tránsito y cálculos dinámicos de rutas.

HU 3.3: Como domiciliario, quiero ver los pedidos que están listos para entrega, con la dirección de destino y la opción de aceptarlos, para organizar mis entregas y llevar los pedidos a tiempo.

Puntuación: 5

Justificación: Requiere una interfaz enfocada a un rol distinto (domiciliario) y crear lógica de bloqueo concurrente para evitar que múltiples personas acepten accidentalmente el mismo encargo.

Total de la épica: 21 puntos de historia.

Épica 4: Integrar un sistema de pagos seguros

Rondas de Planning Poker (Molano / Peraza / Otero / Clavijo / Gómez):

Historia	Ronda 1	Argumentos de la discusión	Ronda 2
HU 4.1	2 / 3 / 3 / 3 / 5	Se separó de la transacción: aquí solo se muestran y se guarda la preferencia, más la bifurcación del pago en efectivo contra entrega.	3
HU 4.2	13 / 13 / 8 / 13 / 20	El 8 no contemplaba los escenarios de falla (rechazos, desconexiones) ni los webhooks. El 20 se bajó a 13 al acordar usar una pasarela con SDK y no certificar el manejo de tarjetas por cuenta propia.	13
HU 4.3	5 / 5 / 3 / 5 / 8	Mostrarlo en pantalla sería un 3, pero la historia pide además correo y SMS: hay que integrar un proveedor externo y maquetar la plantilla.	5

HU 4.1: Como cliente, quiero poder visualizar y seleccionar entre diferentes medios de pago seguros (tarjeta de crédito/débito, transferencia bancaria/PSE o pago en efectivo al recibir), para completar la transacción según mi preferencia.

Puntuación: 3

Justificación: Consiste primordialmente en el diseño visual y recolección de la preferencia de método de pago del usuario, reservando la complejidad de la transacción para el siguiente punto.

HU 4.2: Como cliente, quiero que el pago electrónico sea validado y procesado inmediatamente a través de una pasarela segura antes de confirmar el pedido a la cocina, para asegurar que la transacción fue aprobada.

Puntuación: 13

Justificación: Tarea de muy alta complejidad e incertidumbre. Integrarse con pasarelas financieras requiere manejar tokens de alta seguridad, procesamiento asíncrono (webhooks de confirmación) y manejar escenarios de falla como rechazos bancarios o desconexiones.

HU 4.3: Como cliente, quiero recibir un comprobante de pago por correo electrónico, SMS o en pantalla, para tener un soporte de la transacción realizada.

Puntuación: 5

Justificación: Demanda la conexión y configuración de la aplicación con un proveedor externo de mensajería (ej. un servidor SMTP para correos o un API de SMS) además de maquetar la plantilla del recibo.

Total de la épica: 21 puntos de historia.

Tabla 1. Resumen de la estimación

Épica	Historias	Puntos
Épica 1	6	21
Épica 2	3	18
Épica 3	3	21
Épica 4	3	21
Total	15	81

2. Cálculo de la capacidad del equipo

El equipo calculó su capacidad, es decir el tiempo real que puede dedicar al sprint, a partir de la disponibilidad que los cinco integrantes pueden sostener durante el semestre, ya que todos cursan otras asignaturas.

Integrantes del equipo: 5 estudiantes. El Product Owner y el Scrum Master también desarrollan, como es habitual en un equipo académico; el tiempo que dedican a sus roles se descuenta con el factor de concentración del paso 3.

Duración del sprint: 3 semanas.

Días de trabajo por semana: 4 días (por ejemplo lunes, miércoles, viernes y sábado); los demás días quedan libres para el resto de asignaturas.

Horas diarias por persona: 2 horas.

Paso 1. Días de trabajo en el sprint

3 semanas × 4 días por semana = 12 días de trabajo por persona.

Paso 2. Capacidad máxima teórica (bruta)

12 días × 2 horas/día = 24 horas por persona.

5 personas × 24 horas = 120 horas por sprint.

Paso 3. Factor de concentración (capacidad real)

Del tiempo disponible no todo se convierte en desarrollo: hay ceremonias de Scrum (planeación, revisión, retrospectiva y las reuniones diarias), coordinación entre los cinco e imprevistos académicos. Se aplica un factor de concentración del 80 %, que es el valor recomendado para un equipo sin datos históricos.

Capacidad ajustada = 120 horas × 0,8 = 96 horas por sprint.

Tabla 2. Capacidad del equipo por sprint

Concepto	Cálculo	Resultado
Días de trabajo por persona	3 semanas × 4 días	12 días
Horas por persona	12 días × 2 h/día	24 horas
Capacidad bruta del equipo	5 personas × 24 h	120 horas
Capacidad real (focus factor 80 %)	120 h × 0,8	96 horas

3. Velocidad inicial del equipo y su justificación

Velocidad inicial estimada: 32 puntos de historia por sprint de 3 semanas.

Justificación:

Capacidad base. El equipo dispone de 96 horas reales por sprint, con el factor de concentración ya aplicado.

Esfuerzo integral. La velocidad no mide solo escribir código: el esfuerzo de una historia incluye los requerimientos no funcionales, las pruebas, la corrección de errores, el refactoring y los spikes de investigación técnica.

Métrica de conversión. Se define que 1 punto de historia equivale a 3 horas de trabajo efectivo del equipo, repartidas así:

Codificación e integración: ~1,5 horas (50 %).

Pruebas y validación de los criterios de aceptación: ~0,5 horas.

Spikes e investigación (por ejemplo, la pasarela de pagos o la API de mapas): ~0,5 horas.

Refactoring y corrección de errores: ~0,5 horas.

Cálculo final. 96 horas ÷ 3 horas por punto = 32 puntos de historia por sprint.

Ajuste por incertidumbre. Como es el primer sprint y el equipo no tiene datos históricos, estos 32 puntos son una proyección, no un compromiso cerrado. Al terminar el sprint 1 se reemplazan por la velocidad real (los puntos de las historias que quedaron terminadas) y con ella se vuelve a proyectar el plan.

4. Planeación preliminar de los releases

Los releases son los que el equipo definió en la parte 1 del taller con el mapa de historias: el Release 1 es el producto mínimo viable, con el que un cliente ya puede pedir, pagar y recibir su pizza, y el Release 2 agrega el seguimiento de la entrega y los soportes. El total del backlog son 81 puntos de historia.

Con una velocidad de 32 puntos por sprint, las historias se reparten en tres sprints de 3 semanas. El reparto se hace con historias completas: una historia no se parte entre dos

sprints, porque solo aporta valor cuando está terminada. Por eso los sprints no siempre suman exactamente 32 puntos y la diferencia queda como holgura.

Tabla 3. Duración estimada de cada release

Release	Historias	Puntos	Sprints	Duración acumulada
Release 1 (PMV)	11	55	Sprints 1 y 2	6 semanas
Release 2	4	26	Sprint 3	9 semanas
Total del proyecto	15	81	3 sprints	9 semanas

Tabla 4. Reparto de las historias por sprint

Sprint	Historias	Puntos	Holgura
Sprint 1 Semanas 1 a 3	HU 1.5 (Ver el menú con precios), HU 1.1 (Seleccionar el tamaño de la pizza), HU 1.2a (Agregar productos al carrito), HU 1.2b (Quitar productos o cambiar cantidades), HU 1.3 (Ver el precio total desglosado), HU 1.4 (Elegir entrega a domicilio o para llevar), HU 2.1 (Ver los pedidos pendientes en tiempo real), HU 4.1 (Seleccionar el medio de pago)	32	0
Sprint 2 Semanas 4 a 6	HU 2.2 (Actualizar el estado de un pedido), HU 3.3 (Ver y aceptar pedidos listos para entrega), HU 4.2 (Validar el pago con la pasarela)	23	9
Sprint 3 Semanas 7 a 9	HU 3.1 (Notificar el pedido listo para recoger), HU 3.2 (Rastrear al domiciliario en el mapa), HU 2.3 (Imprimir la factura), HU 4.3 (Recibir el comprobante de pago)	26	6

Lectura del plan:

Sprint 1 (Semanas 1 a 3, 32 puntos). El cliente arma y configura su pedido, y la cocina lo ve llegar.

Sprint 2 (Semanas 4 a 6, 23 puntos). El pedido se paga, cambia de estado y llega al domiciliario: cierra el PMV.

Sprint 3 (Semanas 7 a 9, 26 puntos). Seguimiento de la entrega y soportes: cierra el Release 2.

El Release 1 (PMV) queda terminado al final del sprint 2, es decir en la semana 6, y el Release 2 al final del sprint 3, en la semana 9. La holgura de los sprints 2 y 3 (9 y 6

puntos) no se rellena con más historias a propósito: se reserva para estabilizar, corregir los errores que salgan de las pruebas y preparar la entrega de cada release.

Dependencias que fijan el orden:

• La HU 1.4 (elegir domicilio e ingresar la dirección) va antes que la HU 3.3, porque el domiciliario necesita la dirección de destino.

• La HU 4.1 (seleccionar el medio de pago) va antes que la HU 4.2, que procesa la transacción.

• La HU 2.2 (estados del pedido) va antes que la HU 3.1, porque la notificación se dispara cuando el pedido pasa a "listo para entregar".

• La HU 3.2 (rastreo en el mapa) se deja para el sprint 3: es la de mayor incertidumbre y no bloquea la entrega del PMV.

5. Justificación de la estimación

Por qué puntos y no horas. El equipo no tiene datos históricos de trabajo conjunto, así que estimar en horas sería adivinar. Los puntos de historia son una medida relativa: comparar dos historias entre sí es mucho más fiable que calcular cuánto tarda cada una. Las horas aparecen una sola vez, al convertir la capacidad en velocidad, y esa conversión se corrige con los datos reales del primer sprint.

Escala y referencia. Se usó la serie de Fibonacci porque los saltos crecen con la incertidumbre: entre 1 y 2 hay una diferencia clara, pero discutir si algo grande vale 13 o 14 no aporta nada. La HU 1.1 (2 puntos) sirvió de referencia y las demás se compararon con ella; por eso, por ejemplo, la HU 1.2b (3) es algo mayor que la referencia y la HU 1.2a (5) casi el doble, ya que construye toda la lógica del carrito.

Estimación en equipo. Las cartas se muestran a la vez para que nadie ancle su voto en el de los demás, y los extremos son los que explican. Ese es el valor real de la dinámica: las diferencias de la ronda 1 revelaron supuestos distintos sobre el alcance (si la HU 1.2a incluía editar el carrito, si la HU 2.3 exigía integrar impresoras térmicas, si la HU 4.3 se limitaba a la pantalla) y al aclararlos el consenso salió en la segunda ronda.

Supuestos del cálculo. La estimación de la velocidad depende de cuatro supuestos declarados: 4 días de trabajo por semana, 2 horas por persona y día, un factor de concentración del 80 % y una equivalencia de 3 horas por punto de historia. Si alguno cambia (por ejemplo, en semana de parciales), la velocidad cambia y el plan de releases se recalcula.

Riesgos de la estimación. La incertidumbre se concentra en las dos historias de 13 puntos: la HU 3.2 (rastreo con API de mapas y hora estimada de llegada) y la HU 4.2 (pasarela de pagos). Entre las dos suman 26 de los 81 puntos del backlog, casi un tercio, y ninguna se ha hecho antes en el equipo. Para cada una se reserva un spike de investigación en el sprint anterior a su desarrollo y, si al planear el sprint siguen valiendo 13, se dividen en historias más pequeñas.

Revisión de la estimación. Este plan es preliminar. Al cerrar cada sprint se calcula la velocidad real y se compara con la proyección; con dos o tres sprints el equipo tendrá un promedio confiable y las fechas de los releases se ajustarán con ese dato, no con la estimación inicial.